李林昕

+86 15638867051 Hizan15638867051
linxin.li@mail.utoronto.ca 领英 个人主页

教育背景

多伦多大学 (University of Toronto) **2024 年 9 月 – 2026 年 5 月**
工程硕士，应用机器学习方向；*GPA: 3.9/4.0* 多伦多，加拿大

英属哥伦比亚大学 (University of British Columbia) **2021 年 9 月 – 2024 年 5 月**
应用科学学士，计算机工程专业；以优异成绩毕业；*GRE: 336/340* 温哥华，加拿大

技术技能

编程语言与前端: Python, JS/TS, Java, C/C++, Go, SQL, HTML/CSS, React, Node.js, FastAPI, Flask, RESTful API
云原生与 DevOps: AWS (Certified), Docker, K8s, Jenkins, GitHub Actions, CI/CD, Microservices, Linux, Git
数据、AI 与工具: MongoDB, MySQL, ETL, EDA, OpenAI, Hugging Face, LLM Integration, Postman, Jira, DBeaver

工作与科研经历

SAP (思爱普) **2025 年 4 月 – 2025 年 9 月**
AI 应用工程师（实习） 上海，中国

- 主导 Joule AI 助手核心功能开发；基于 Python 与 GPT-5 研发涵盖 RAG 检索、多插件及促销计划生成的多个核心功能；架构基于 Project Piper 与 Kyma 的模块化 CI/CD 流水线，利用 ArgoCD 实现云原生应用的自动化部署。
- 设计并实施基于 Cucumber 与 Pytest 的全链路（E2E）测试框架，构建 Jenkins 自动化质量度量体系；在高频迭代中确保集成测试覆盖率稳定在 90%+，单元测试覆盖率达 95%+。
- 采用 Flask 与 C/C++ 混合编程实现 50+ IoT 设备与 OrionStar 机器人的协同调度系统；攻克高并发同步控制难题，实现复杂演示场景的全自动化流水线作业，系统稳定性提升显著。

城市数据研究中心 (Urban Data Research Center) **2024 年 9 月 – 2025 年 4 月**
全栈开发工程师（兼职） 多伦多，加拿大

- 自主设计并实现 DCR (Divide-Conquer-Review) 自动化提取框架：采用“分治-抽取-交叉校验”三阶段架构解决长文档处理难题；集成 Python 与 GPT 能力，实现对 1.4w+ 复杂数据点的高精度提取（准确率 88%）。
- 主导“城市数字孪生”项目 React 交互式看板开发，通过 SPARQL 协议深度集成 GraphDB 知识图谱；实现分层城市指标地图及可视化图表，支持数据驱动的城市规划。

英属哥伦比亚大学 (UBC) **2023 年 9 月 – 2024 年 4 月**
研究助理 | 由 *ECE* 系教授指导 温哥华，加拿大

- 主导基于 Ubuntu/Linux 环境的深度学习架构优化；通过精细化管理 NVIDIA GPU 硬件资源与计算驱动配置，在受限资源下将模型训练准确率提升至 85%。
- 搭建联邦学习 (Federated Learning) 分布式模拟环境；设计并实现 Bash 自动化测试套件，协调大规模训练周期并自动化注入对抗攻击。

顺丰速运 (SF Express) **2023 年 4 月 – 2023 年 9 月**
软件开发工程师（实习） 深圳，中国

- 研发快递服务合规性 LLM 自动化评估流水线：针对海量语音转写数据构建多维 Ground-truth 基准，设计并实现了一套涵盖多标签违规检测的标准化评估框架；通过与模型迭代流程深度集成，实现模型更新后的全自动性能评测与差异分析。
- 开发多维度业务合规管理看板，支持区域经理对快递员作业规范进行实时监控；设计层级数据查询 API，支持百万级数据下钻分析与复杂逻辑过滤。
- 基于 Selenium 与 Pytest 构建企业级自动化测试套件；覆盖 300 余项核心业务功能点，通过 CI 流程确保了高频迭代下的系统稳定性。

重点项目

基于 RAG 架构的多模态智能教科书学习平台 | *Python, Javascript, Gemini/GPT/Deepseek, React* **2026 年 1 月 – 至今**

- 独立开发 AI 辅助学习项目：打造集“文档解析-向量检索-内容生成-多模态输出”于一体的智能学习工具，实现对 PDF 教科书的深度智能化拆解；通过 Python 核心逻辑与 React 前端交互，为用户提供沉浸式的讲解与实时测验体验。
- 技术栈整合与性能优化：选型并优化了跨模型的 API 调度逻辑，引入 Docker 容器化方案降低部署门槛；基于 ChromaDB 向量数据库实现海量知识点的持久化与检索优化；封装多模型驱动接口，通过注入语义上下文（Context Injection）显著提升了 LLM 对教科书知识点的回答准确度。